

Module : Apprentissage Automatique

Filière : BDIA'1 — Année universitaire 2025-2026

Chapitre 4

Méthodes d'apprentissage non supervisé

Prof. Dr Naoual Attaoui

Dr Naoual Attaoui

— 2 —

Chapitre 4 : Méthodes d'apprentissage non supervisé

Table des matières

1. Introduction
2. Clustering
3. K-Means
4. Clustering hiérarchique
5. DBSCAN
6. Réduction de dimensionnalité
7. Détection d'anomalies
8. Implémentation Python

1. Introduction

L'apprentissage non supervisé est une branche fondamentale du *machine learning* qui se distingue par l'absence d'étiquettes dans les données d'entraînement. Contrairement à l'apprentissage supervisé où chaque observation est associée à une valeur cible connue (classe ou valeur numérique), l'apprentissage non supervisé travaille uniquement avec les variables d'entrée, sans information sur la sortie attendue.

Ces méthodes cherchent par exemple à regrouper automatiquement des observations semblables, à détecter des structures latentes, à compresser l'information, à visualiser des données de grande dimension, ou encore à signaler des points atypiques. Elles jouent ainsi un rôle central dans l'exploration des données, l'analyse descriptive et la préparation de modèles plus avancés.

En pratique, l'apprentissage non supervisé est particulièrement utile lorsque l'étiquetage est coûteux, long ou impossible. C'est souvent le cas dans des bases de données industrielles, médicales, financières ou commerciales où l'on possède une grande quantité de mesures, mais sans annotation experte pour chaque observation.

Les applications sont très variées : segmentation de clients en marketing, détection d'anomalies en cybersécurité, compression d'images, systèmes de recommandation, analyse génomique, réduction de dimensionnalité pour la visualisation, ou encore regroupement de documents textuels.

On distingue principalement trois grandes familles de méthodes :

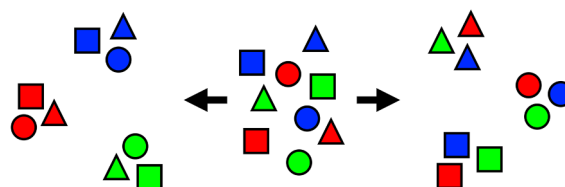
- **Le clustering (regroupement)** : partitionner les données en groupes homogènes appelés clusters.
- **La réduction de dimensionnalité** : représenter les données avec moins de variables tout en préservant l'information essentielle.
- **La détection d'anomalies** : repérer les observations rares, isolées ou peu cohérentes avec l'ensemble du nuage.

Ce chapitre présente les algorithmes fondamentaux de chacune de ces familles, en insistant sur l'intuition, la formulation mathématique, les critères d'évaluation et la mise en œuvre pratique en Python.

✉ **Fil conducteur — Segmentation clients** : Tout au long de ce chapitre, nous utiliserons la segmentation de clients d'un site e-commerce comme exemple fil conducteur. À partir de variables comme le revenu annuel, le score de dépenses et l'âge, l'objectif est de découvrir des profils de clients sans information préalable sur les groupes.

2. Clustering

Le clustering, ou regroupement non supervisé, consiste à organiser un ensemble d'observations en groupes appelés **clusters**, de sorte que les observations d'un même cluster soient plus similaires entre elles que celles appartenant à des clusters différents.



Selon la méthode utilisée, cette affectation peut être :

- **dure** : chaque observation appartient à un seul cluster ;
- **souple** ou **probabiliste** : chaque observation peut appartenir à plusieurs clusters avec des degrés d'appartenance différents.

2.1. Formulation du problème de partitionnement

Soit un ensemble de m observations $X = \{x_1, x_2, \dots, x_m\}$ dans un espace à n dimensions. Dans le cas d'un **partitionnement dur**, le problème consiste à trouver une partition $P = \{C_1, C_2, \dots, C_k\}$ de X en K sous-ensembles non vides, vérifiant :

- **Exhaustivité** : chaque observation appartient à au moins un cluster : $C_1 \cup C_2 \cup \dots \cup C_k = X$.
- **Exclusivité mutuelle** : une observation ne peut pas appartenir à deux clusters différents : $C_i \cap C_j = \emptyset$ pour $i \neq j$.
- **Non-vacuité** : aucun cluster n'est vide : $C_k \neq \emptyset$ pour tout $k = 1, \dots, K$.

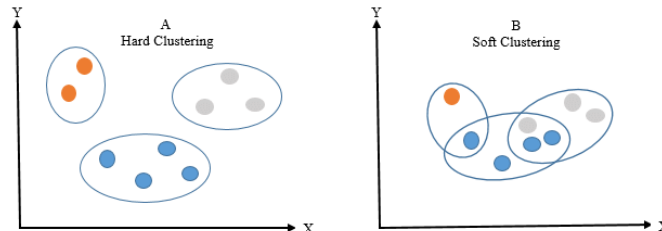
Partition dure vs partition souple

Dans une **partition dure** (*hard clustering*), chaque observation est assignée à un seul cluster. C'est le cas de K-Means, de la CAH et de DBSCAN.

Dans une **partition souple** (*soft clustering* ou *fuzzy clustering*), chaque observation possède un degré d'appartenance à chaque cluster, exprimé sous forme de probabilité. C'est le cas des modèles de mélange gaussien (GMM) où l'on obtient :

$$P(x_i \in C_k) \in [0, 1] \text{ et } \sum_k P(x_i \in C_k) = 1$$

Le choix entre partition dure et souple dépend du problème : la partition souple est préférable lorsqu'une observation peut naturellement appartenir à plusieurs groupes (par exemple, un client ayant un profil intermédiaire entre deux segments).



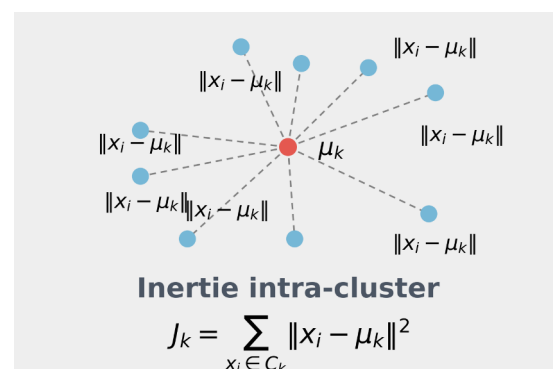
Objectif d'optimisation

La plupart des algorithmes de partitionnement cherchent à optimiser un critère de qualité. Le plus courant est la minimisation de l'**inertie intra-cluster** (WCSS : **Within-Cluster Sum of Squares**) :

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

où :

- K : nombre de clusters,
- C_k : ensemble des points du cluster k ,
- x_i : un point de données,
- μ_k : centroïde du cluster k .



Ce problème d'optimisation est NP-difficile dans le cas général, ce qui explique pourquoi les algorithmes comme K-Means utilisent des heuristiques itératives qui convergent vers un minimum local (l'algorithme de Lloyd le plus souvent).

2.2. Notion de similarité et distance

Le choix d'une mesure de distance est crucial dans tout algorithme de clustering. Les distances les plus utilisées sont :

Distance euclidienne : distance géométrique classique dans l'espace à n dimensions.

$$d(x, y) = \sqrt{(\sum_{i=1}^n (x_i - y_i)^2)}$$

Distance de Manhattan : somme des valeurs absolues des différences composante par composante.

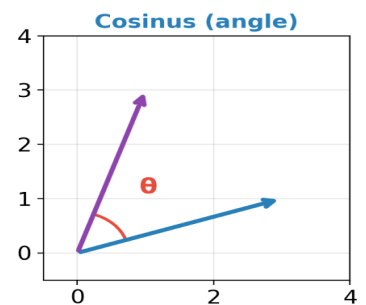
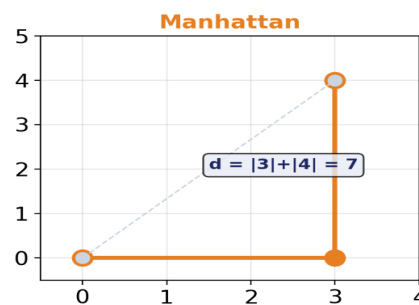
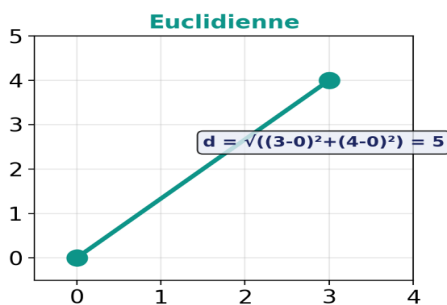
$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

Distance de Minkowski : généralisation avec un paramètre p .

$$d(x, y) = (\sum_{i=1}^n |x_i - y_i|^p)^{1/p}$$

Similarité cosinus : mesure l'angle entre deux vecteurs, utilisée pour les données textuelles.

$$\text{sim}(x, y) = (x \cdot y) / (\|x\| \cdot \|y\|)$$



Il est souvent recommandé de **standardiser** les données avant d'appliquer un algorithme de clustering pour éviter qu'une variable à grande échelle domine les calculs de distance.

2.3. Types de clustering

On distingue plusieurs approches :

Méthode	Principe	Exemple
Partitionnement	Découpe en K groupes disjoints	K-Means, K-Médoïdes
Hiérarchique	Construction d'un arbre de clusters	CAH (Ward)
Basé sur la densité	Groupes = zones denses séparées par des zones creuses	DBSCAN
Probabiliste	Modèle de mélange (partition souple)	GMM (EM)

2.4. Evaluation du clustering

Évaluer un clustering est plus délicat qu'en apprentissage supervisé, car on ne dispose généralement pas de **vérité terrain**. On distingue deux grandes catégories de métriques : les **métriques internes**, qui ne nécessitent pas d'étiquettes, et les **métriques externes**, qui comparent le clustering obtenu à une partition de référence lorsqu'elle existe (ne sont pas détaillées dans ce chapitre).

Métriques internes (sans étiquettes)

Métrique	Définition	Interprétation
Inertie (WCSS)	Somme des distances ² au centroïde	Plus faible = plus compact
Score silhouette	Cohésion vs séparation	Proche de 1 = bon clustering
Indice de Davies-Bouldin	Ratio dispersion / séparation	Plus faible = meilleur
Indice de Calinski-Harabasz	Variance inter / intra	Plus élevé = meilleur

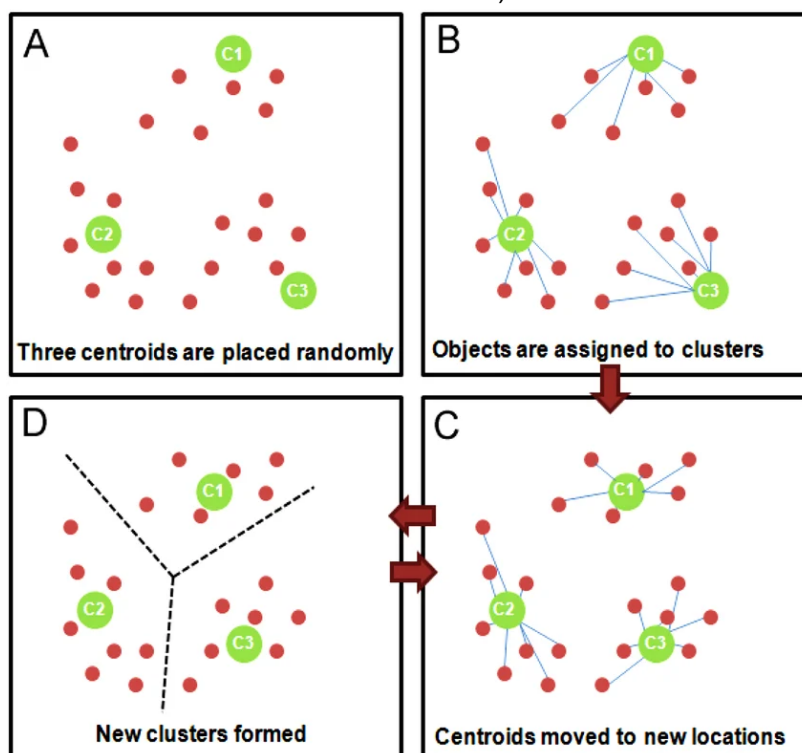
3. K-Means

3.1. Principe de l'algorithme

L'algorithme K-Means est l'un des algorithmes de clustering les plus populaires. Il vise à partitionner un ensemble de m observations en K clusters, où chaque observation est affectée au cluster dont le **centroïde** (centre de gravité) est le plus proche. L'objectif est de minimiser l'**inertie**.

3.2. Étapes de l'algorithme

1. **Initialisation** : choisir aléatoirement K points comme centroïdes initiaux.
2. **Affectation** : chaque observation est assignée au cluster dont le centroïde est le plus proche (distance euclidienne).
3. **Mise à jour** : recalculer le centroïde de chaque cluster comme la moyenne de ses observations.
4. **Convergence** : répéter les étapes 2 et 3 jusqu'à ce que les affectations ne changent plus (ou qu'un nombre maximal d'itérations soit atteint).



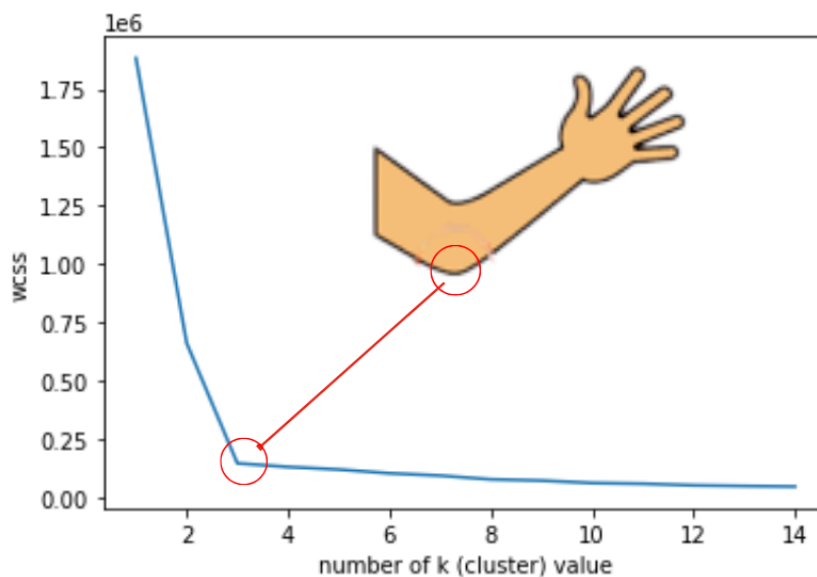
✉ **Fil conducteur — Segmentation clients** : En appliquant K-Means avec $K=4$ sur les données clients (revenu annuel, score de dépenses), l'algorithme découvre quatre profils : « économes », « dépensiers », « modérés » et « premium ». Chaque client est automatiquement affecté au groupe le plus proche de son profil.

3.3. Choix du nombre de clusters K

K n'est pas déterminé automatiquement par l'algorithme. Deux méthodes classiques permettent de le choisir :

Méthode du coude (Elbow method)

On exécute K-Means pour différentes valeurs de K et on trace l'inertie en fonction de K . Le « coude » de la courbe suggère le nombre optimal de clusters : c'est le point où la diminution de l'inertie ralentit significativement.

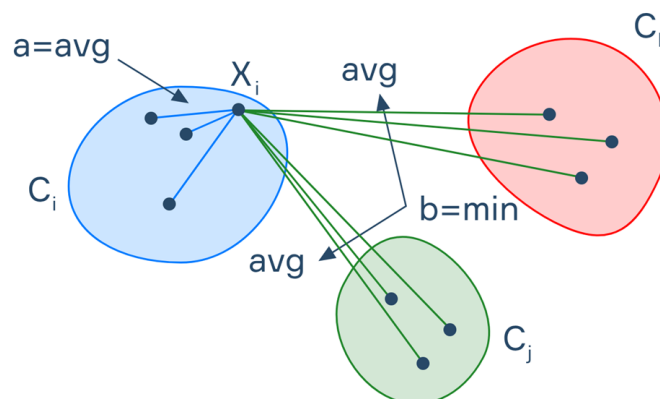


Score silhouette

Le coefficient de silhouette mesure la qualité du clustering pour chaque observation i :

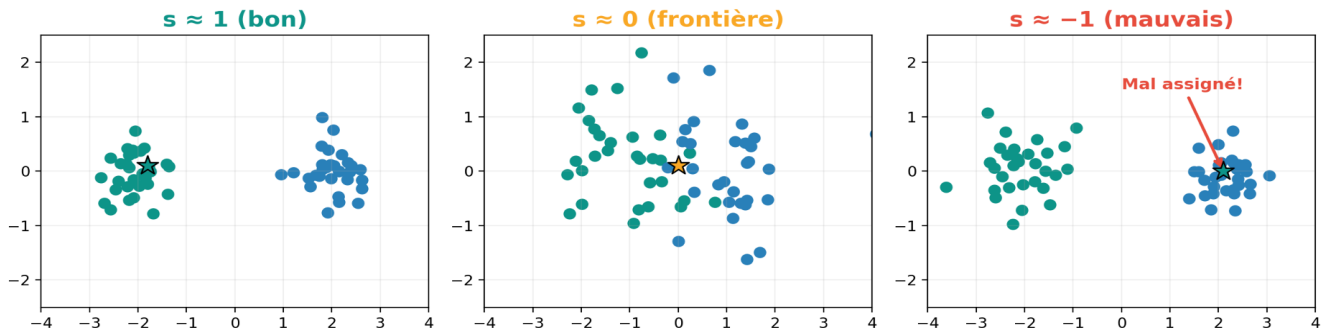
$$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$$

où $a(i)$ est la distance moyenne intra-cluster et $b(i)$ la distance moyenne au cluster le plus proche.



Le coefficient de silhouette varie entre -1 et 1 :

- une valeur proche de 1 indique que l'observation est bien affectée à son cluster ;
- une valeur proche de 0 indique qu'elle se situe à la frontière entre deux clusters ;
- une valeur proche de -1 suggère une mauvaise affectation.



En pratique, pour choisir K , on calcule le **score de silhouette moyen** pour plusieurs valeurs de K , puis on retient généralement celle qui maximise ce score, sous réserve qu'elle reste cohérente avec l'interprétation des données.

3.4. Avantages et limites de K-Means

Avantages	Limites
Simple et rapide	Nécessite de fixer K à l'avance
Passe à l'échelle (grands jeux de données)	Sensible à l'initialisation des centroïdes
Facile à interpréter	Ne détecte que des clusters sphériques
Converge rapidement	Sensible aux valeurs aberrantes (outliers)

Remarque : L'algorithme **K-Means++** améliore l'initialisation en choisissant les centroïdes initiaux de manière plus éloignée les uns des autres, réduisant le risque de convergence vers un minimum local. C'est l'initialisation par défaut dans scikit-learn.

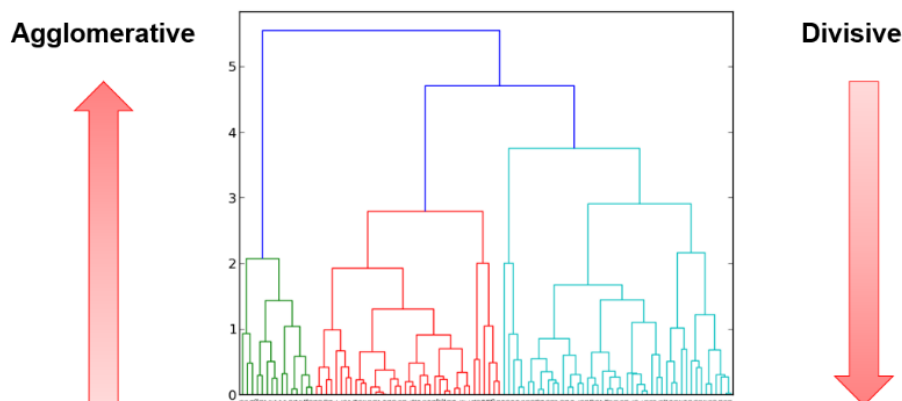
4. Clustering hiérarchique

4.1. Principe

Le clustering hiérarchique construit une hiérarchie de clusters, représentée sous forme d'un arbre appelé **dendrogramme**. Contrairement à K-Means, cette méthode ne nécessite pas de fixer le nombre de clusters à l'avance.

On distingue deux approches :

- **Agglomérative (ascendante)** : chaque observation commence comme son propre cluster, puis les clusters les plus proches sont fusionnés successivement.
- **Divisive (descendante)** : on part d'un seul cluster contenant toutes les observations, puis on le divise récursivement.

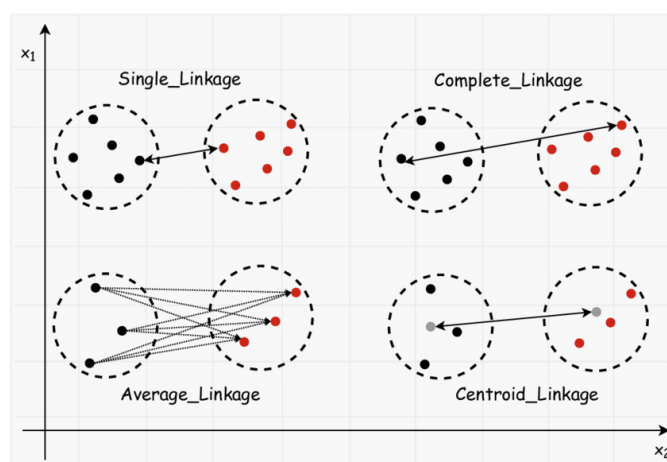


En pratique, l'approche **agglomérative** (CAH — Classification Ascendante Hiérarchique) est la plus courante.

4.2. Critères de liaison (linkage)

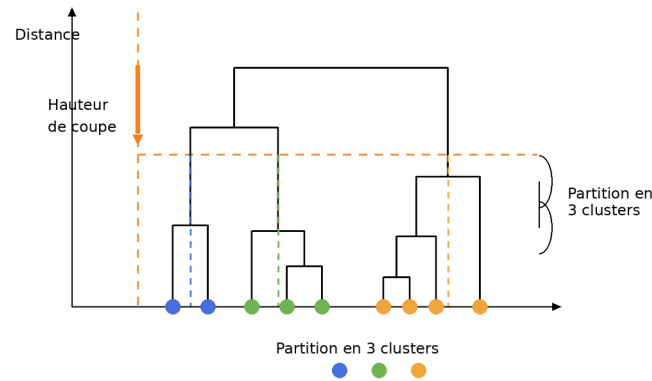
Le critère de liaison détermine comment la distance entre deux clusters est calculée :

Critère	Définition	Caractéristiques
Single linkage	Distance minimale entre paires	Sensible au bruit, produit des chaînes
Complete linkage	Distance maximale entre paires	Clusters compacts, sensible aux outliers
Average linkage	Moyenne des distances entre paires	Compromis entre single et complete
Centroid linkage	Distance entre les centroïdes des clusters	Simple et intuitif, mais peut produire des inversions dans le dendrogramme
Ward	Minimise l'augmentation de l'inertie	Clusters sphériques et équilibrés



4.3. Dendrogramme et découpage

Le dendrogramme est une représentation graphique de la hiérarchie des fusions. L'axe vertical représente la distance à laquelle les clusters sont fusionnés. Pour obtenir une partition en K clusters, on « coupe » le dendrogramme à un niveau de distance choisi. La hauteur de coupe correspond au compromis entre le nombre de clusters et l'homogénéité des groupes.



4.4. Avantages et limites

- **Avantages** : pas de choix de K à l'avance ; fournit une vue d'ensemble via le dendrogramme ; déterministe.
- **Limites** : complexité élevée ($O(m^2 \log m)$ ou $O(m^3)$) ; ne passe pas à l'échelle pour de très grands jeux de données ; les fusions sont irréversibles.

5. DBSCAN

5.1. Principe

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) est un algorithme de clustering basé sur la densité. Il définit les clusters comme des régions de haute densité, séparées par des régions de faible densité. Contrairement à K-Means, DBSCAN ne nécessite pas de spécifier le nombre de clusters et peut détecter des clusters de forme arbitraire.

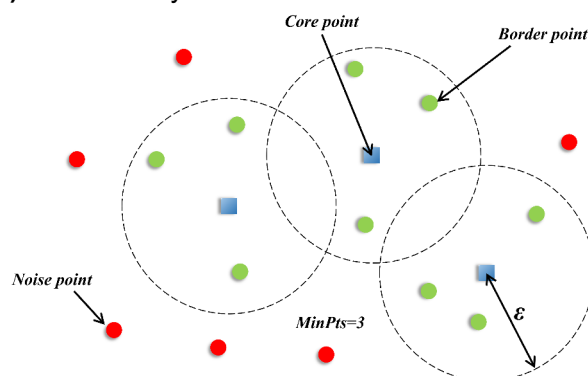
5.2. Paramètres et définitions

DBSCAN repose sur deux paramètres :

- ϵ (epsilon) : le rayon de voisinage autour de chaque point.
- **MinPts** : le nombre minimum de points requis dans le voisinage pour qu'un point soit un **point noyau**.

L'algorithme classe chaque point en trois catégories :

- **Point noyau (core point)** : possède au moins MinPts voisins dans un rayon ϵ .
- **Point frontière (border point)** : n'est pas un point noyau mais se trouve dans le voisinage d'un point noyau.
- **Point bruit (noise)** : n'est ni noyau ni frontière. Il est considéré comme un outlier.



5.3. Algorithme

L'algorithme parcourt chaque point non visité :

1. Si le point est un point noyau, un nouveau cluster est créé et tous les points accessibles par densité sont ajoutés.
2. Si le point n'est pas un point noyau et se trouve dans le voisinage d'un cluster existant, il est ajouté comme point frontière.
3. Sinon, le point est étiqueté comme bruit.

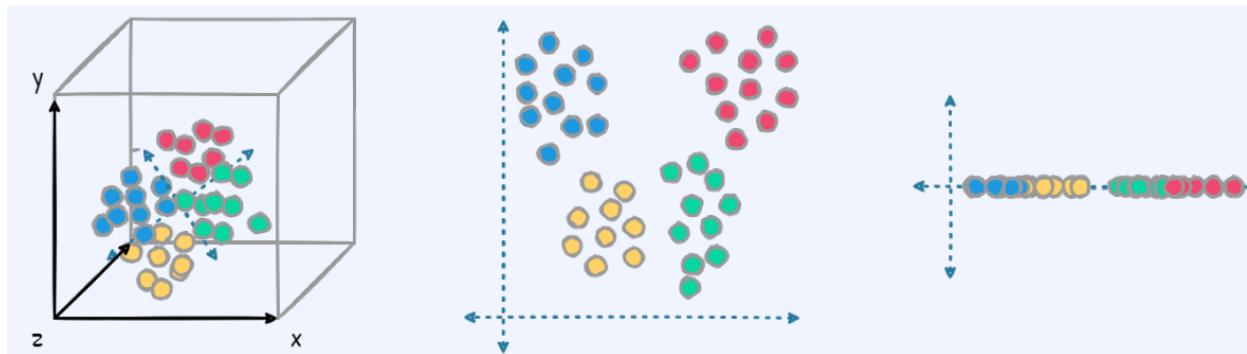
5.4. Avantages et limites

Avantages	Limites
Pas besoin de fixer K	Sensible au choix de ϵ et MinPts
Détecte des clusters de forme arbitraire	Difficultés avec des densités variables
Robuste aux outliers (les étiquette comme bruit)	Coût élevé en grande dimension
Déterministe (hors points frontières)	Moins performant en très haute dimension

✉ **Fil conducteur — Segmentation clients** : DBSCAN appliqué aux données clients (après standardisation) identifie automatiquement les groupes denses et étiquette les clients atypiques comme du bruit, ce qui est utile pour détecter des comportements inhabituels.

6. Réduction de dimensionnalité

La réduction de dimensionnalité consiste à projeter les données d'un espace de grande dimension vers un espace de dimension plus faible, tout en préservant au maximum l'information pertinente.



Cette étape est souvent nécessaire pour :

- **Visualiser** les données en 2D ou 3D.
- **Réduire le bruit** en éliminant les dimensions peu informatives.
- **Améliorer les performances** des modèles en limitant *la malédiction de la dimension*.
- **Accélérer** l'entraînement en réduisant le nombre de variables.

6.1. Analyse en Composantes Principales (ACP / PCA)

6.1.1. Principe

L'Analyse en Composantes Principales (ACP), ou *Principal Component Analysis* (PCA), est la méthode de réduction de dimensionnalité la plus utilisée. Elle cherche les directions (appelées **composantes principales**) de variance maximale dans les données.

L'idée est de projeter les données sur un sous-espace de dimension $d < n$ tel que la variance expliquée soit maximale. Les composantes principales sont les vecteurs propres de la matrice de covariance, classés par valeur propre décroissante.

6.1.2. Formulation mathématique

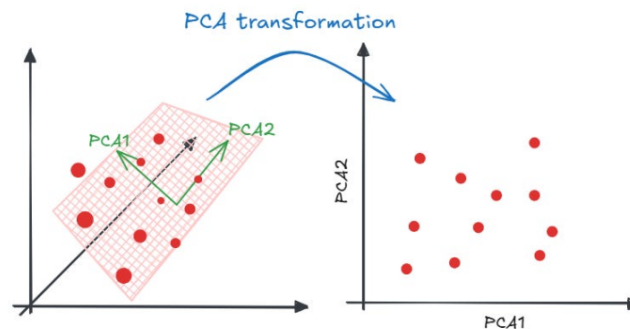
Soit X la matrice des données centrées ($m \times n$). La matrice de covariance est :

$$C = (1/m) X^T X$$

On cherche les vecteurs propres $v_1, v_2, \dots, v_n$ de C associés aux valeurs propres $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. Tel que $C v_i = \lambda_i v_i$. Les d premiers vecteurs propres forment la matrice de projection W :

$$Z = X \cdot W$$

où Z est la représentation réduite de dimension $m \times d$.



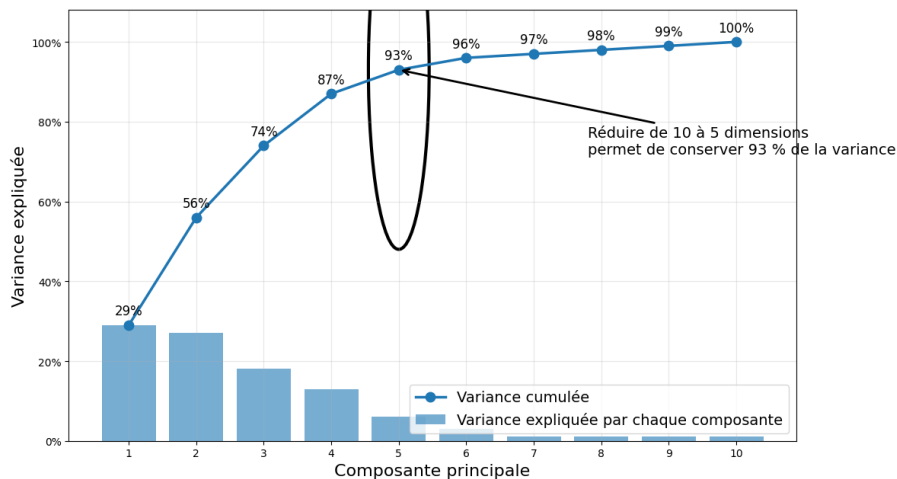
6.1.3. Variance expliquée

La proportion de variance expliquée par la k -ième composante est :

$$VE_k = \lambda_k / \sum_{i=1}^n \lambda_i$$

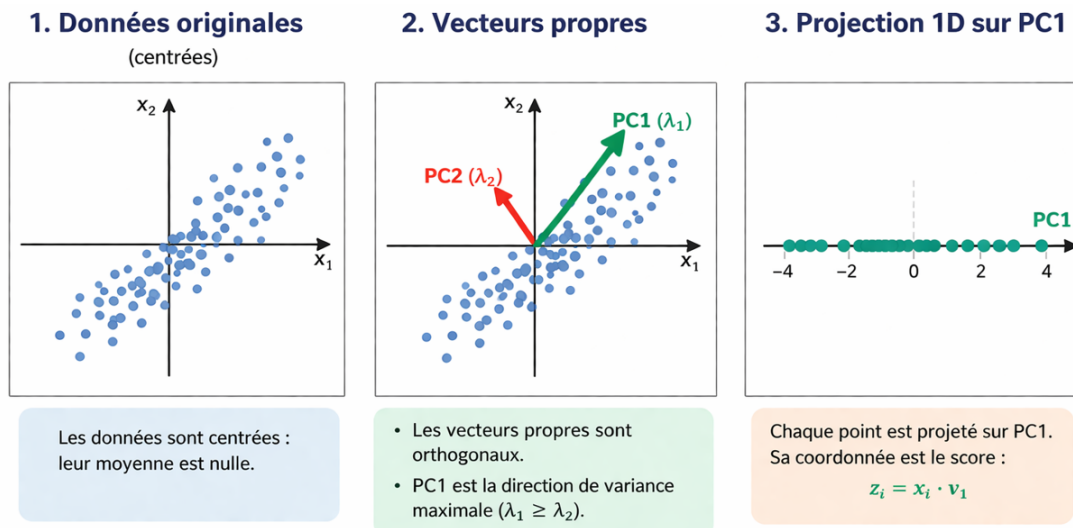
En pratique, on choisit d tel que la variance cumulée atteigne un seuil (par exemple 95 %).

Graphique de la variance expliquée cumulée pour l'ACP



6.1.4. Étapes de l'ACP

1. Centrer (et éventuellement standardiser) les données.
2. Calculer la matrice de covariance.
3. Calculer les valeurs propres et vecteurs propres.
4. Trier les vecteurs propres par valeur propre décroissante.
5. Sélectionner les d premiers vecteurs propres.
6. Projeter les données sur le sous-espace.



Remarque : En ACP, on garde les vecteurs propres associés aux plus grandes valeurs propres. La projection sur PC1 conserve le plus de variance possible.

✉ **Fil conducteur — Segmentation clients :** En appliquant l'ACP sur les 5 variables client, on réduit à 2 composantes principales qui expliquent 85 % de la variance, permettant de visualiser les clusters en 2D.

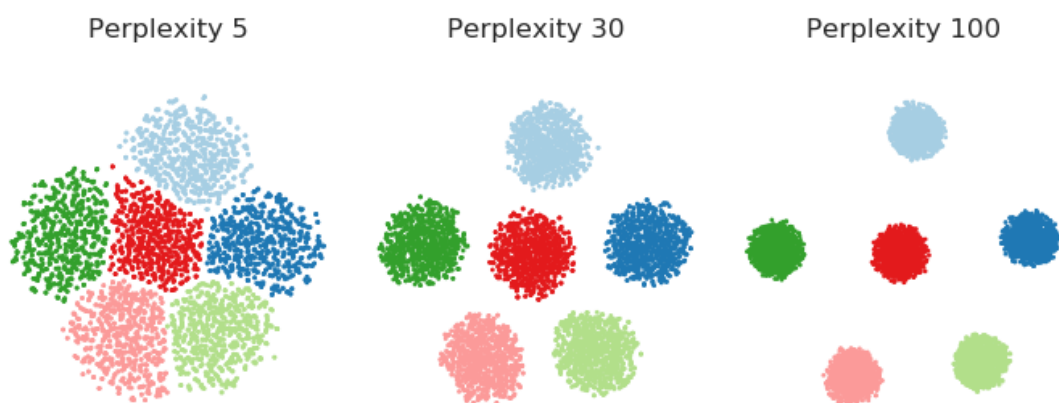
6.2. t-SNE

Le t-SNE (*t-distributed Stochastic Neighbor Embedding*) est une technique non linéaire de réduction de dimensionnalité, particulièrement adaptée à la **visualisation** de données en haute dimension.

Contrairement à l'ACP qui préserve la structure globale, t-SNE cherche à préserver les **distances locales** : deux points proches dans l'espace original doivent rester proches dans la projection. Le principe repose sur la minimisation de la divergence de Kullback-Leibler entre les distributions de probabilité dans les deux espaces.

Points importants :

- t-SNE est principalement un outil de visualisation, pas de prétraitement.
- Le paramètre **perplexité** contrôle l'équilibre entre structure locale et globale (valeurs typiques : 5–50).
- Les résultats peuvent varier selon l'initialisation aléatoire.
- Coût de calcul élevé pour de très grands jeux de données.



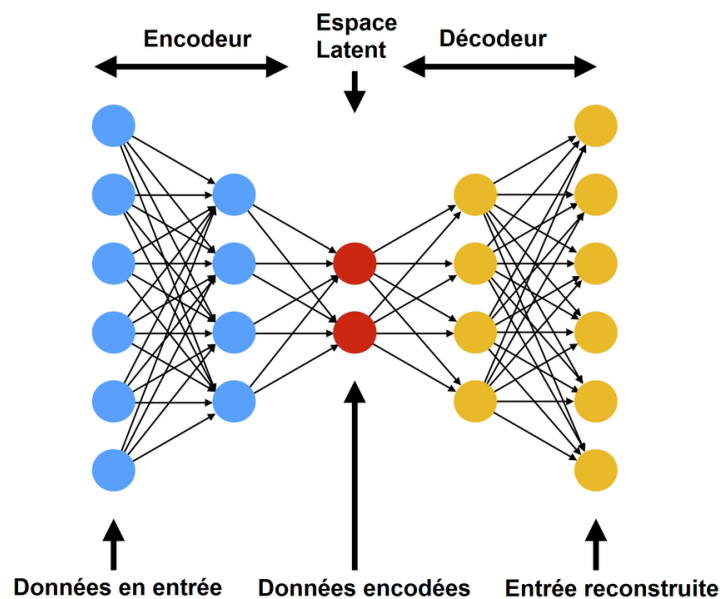
6.3. Auto-encodeurs

Les auto-encodeurs constituent une approche de réduction de dimensionnalité basée sur les réseaux de neurones. Contrairement à l'ACP qui impose une transformation linéaire, les auto-encodeurs peuvent apprendre des représentations **non linéaires** des données.

6.3.1. Architecture générale

Un auto-encodeur est composé de deux parties :

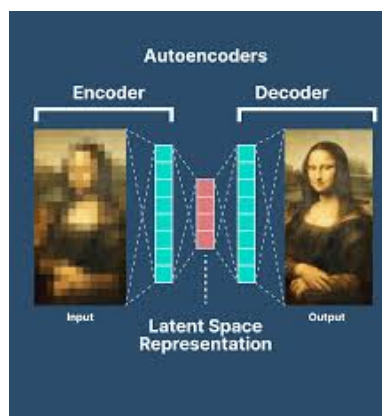
- **L'encodeur** : transforme l'entrée x en une représentation compressée z de dimension inférieure (appelée **espace latent** ou *bottleneck*) : $z = f(x)$.
- **Le décodeur** : reconstruit les données à partir de z : $\hat{x} = g(z)$.



Le modèle est entraîné à minimiser l'**erreur de reconstruction** :

$$L = \|x - \hat{x}\|^2 = \|x - g(f(x))\|^2$$

L'idée clé est que si le réseau parvient à reconstruire fidèlement x à partir d'une représentation z de dimension plus faible, alors z capture l'essentiel de l'information contenue dans les données.



Remarque : Lorsque l'encodeur et le décodeur sont des fonctions linéaires (sans fonction d'activation), l'auto-encodeur retrouve la même solution que l'ACP. Les auto-encodeurs sont donc une **généralisation non linéaire** de l'ACP.

6.4. Comparaison des méthodes de réduction

Critère	ACP	t-SNE	Auto-encodeur
Projection	Linéaire	Non linéaire	Non linéaire
Préserve	Variance globale	Voisinages locaux	Structure apprise
Déterministe	Oui	Non	Non
Utilisation	Réduction + prétrait.	Visualisation	Réduction + génération
Interprétabilité	Élevée	Faible	Faible

7. Détection d'anomalies

La détection d'anomalies (ou *outlier detection*) est une application majeure de l'apprentissage non supervisé. Elle consiste à identifier les observations qui dévient significativement du comportement « normal » des données, sans disposer d'étiquettes indiquant quelles observations sont anormales.

Les applications sont nombreuses : détection de fraudes bancaires, intrusions réseau, défauts de fabrication, anomalies médicales, ou encore comportements inhabituels de clients.

7.1. Isolation Forest

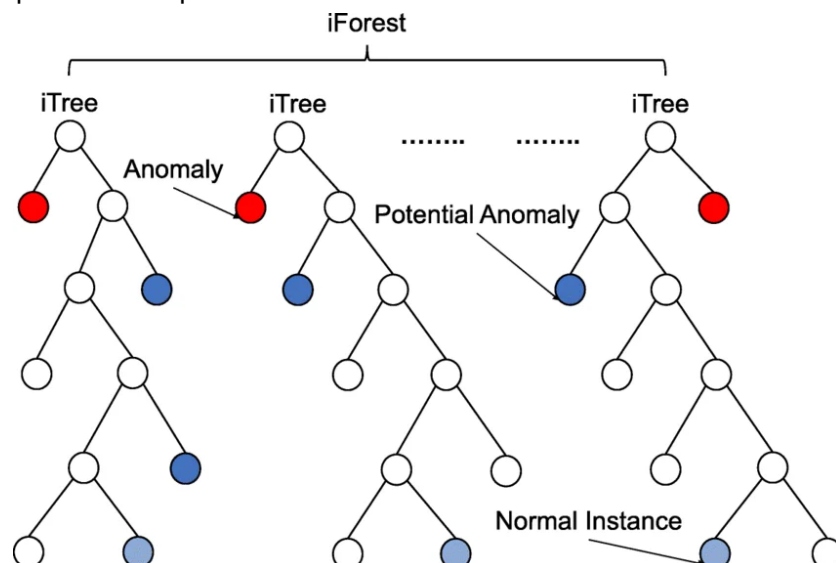
L'**Isolation Forest** repose sur une intuition simple : les anomalies sont des points **rare** et **différents**, donc plus faciles à isoler que les points normaux.

Principe

L'algorithme construit un ensemble d'arbres de décision aléatoires (*isolation trees*). À chaque nœud, une variable et un seuil de séparation sont choisis aléatoirement. Les anomalies, étant isolées, nécessitent en moyenne moins de partitions pour être séparées du reste des données.

Le **score d'anomalie** d'une observation est basé sur la profondeur moyenne à laquelle elle est isolée dans l'ensemble des arbres : $s(x) = 2^{-E[h(x)]} / c(m)$

où $E[h(x)]$ est la profondeur moyenne d'isolation et $c(m)$ un facteur de normalisation. Un score proche de 1 indique une forte probabilité d'anomalie.



Avantages

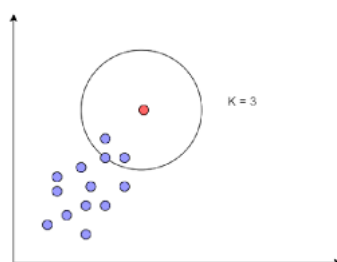
- Très rapide et efficace, même sur de grands jeux de données.
- Ne nécessite pas de modéliser la distribution des données normales.
- Fonctionne bien en haute dimension.

7.2. Local Outlier Factor (LOF)

Le LOF (*Local Outlier Factor*) est une méthode de détection d'anomalies basée sur la **densité locale**. L'idée est qu'une anomalie se trouve dans une région de densité significativement plus faible que ses voisins.

Pour chaque point, le LOF compare sa densité locale (estimée via la distance à ses k plus proches voisins) à la densité locale de ses voisins. Un LOF nettement supérieur à 1 signale une anomalie.

Avantage principal : le LOF détecte les anomalies *locales*, c'est-à-dire des points qui sont anormaux par rapport à leur voisinage, même si leur position absolue ne semble pas extrême.



7.3. Détection par auto-encodeur

Les auto-encodeurs peuvent également servir à la détection d'anomalies. Le principe est le suivant :

1. Entraîner l'auto-encodeur sur des données supposées normales.
2. Calculer l'erreur de reconstruction pour chaque nouvelle observation.
3. Les observations dont l'erreur de reconstruction dépasse un seuil sont considérées comme des anomalies.

L'intuition est que l'auto-encodeur apprend à bien reconstruire les données normales. Les anomalies, ayant une distribution différente, seront mal reconstruites, produisant une erreur élevée.

7.4. Comparaison des méthodes

Méthode	Approche	Avantage principal	Limite
Isolation Forest	Isolement aléatoire	Rapide, scalable	Moins précis en densité variable
LOF	Densité locale	Détecte anomalies locales	Sensible au choix de k
DBSCAN (bruit)	Densité globale	Clustering + anomalies	Sensible à ϵ et MinPts
Auto-encodeur	Reconstruction	Données complexes	Nécessite entraînement

✉ **Fil conducteur — Segmentation clients** : En appliquant Isolation Forest sur les données clients, on identifie environ 5 % de clients atypiques : certains ont un revenu très élevé mais un score de dépenses très faible, ou inversement. Ces clients méritent une attention particulière (offres ciblées, vérification de fraude, etc.).

8. Implémentation Python

Cette section présente une mise en œuvre en Python des principales méthodes d'apprentissage non supervisé étudiées dans ce chapitre. L'objectif est de montrer comment, à partir d'un jeu de données réel, on peut construire une chaîne complète d'analyse comprenant la préparation des données, le clustering, la réduction de dimensionnalité, la visualisation et la détection d'anomalies.

Le jeu de données utilisé est **Mall Customers**, souvent employé pour illustrer la segmentation de clients. Il contient des variables simples et interprétables, notamment l'âge, le revenu annuel et le score de dépenses.

8.1. Chargement et préparation des données

La première étape consiste à charger le jeu de données, à sélectionner les variables quantitatives utiles et à les standardiser. Cette standardisation est importante, car plusieurs méthodes non supervisées reposent sur des calculs de distance. Sans cette étape, une variable de grande amplitude pourrait dominer les autres et fausser l'analyse.

```
# =====
# 8.1. Chargement et préparation des données
# =====

# Bibliothèques de base
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Outil de standardisation
from sklearn.preprocessing import StandardScaler

# -----
# 1) Chargement du jeu de données
# -----
# On lit directement un fichier CSV disponible en ligne.
# On peut télécharger une fois le fichier CSV puis de remplacer l'URL par un nom de fichier local, par
# exemple:
# pd.read_csv("Mall_Customers.csv").
url = "https://raw.githubusercontent.com/tirthajyoti/Machine-Learning-
with-Python/master/Datasets/Mall_Customers.csv"
df = pd.read_csv(url)

# -----
# 2) Aperçu des données
# -----
# head() affiche les premières lignes
print(df.head())

# info() donne les types des colonnes et le nombre de valeurs non nulles
print(df.info())

# describe() donne un résumé statistique des variables numériques
print(df.describe())

# -----
# 3) Sélection des variables quantitatives
# -----
# On garde uniquement les colonnes numériques utiles au clustering.
X = df[["Age", "Annual Income (k$)", "Spending Score (1-100)"]]

# -----
# 4) Standardisation
# -----
# StandardScaler transforme chaque variable pour qu'elle ait :
# - une moyenne proche de 0
# - un écart-type proche de 1
# Cela évite qu'une variable de grande amplitude domine les autres.
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
CustomerID Gender Age Annual Income (k$) Spending Score (1-100)
0 1 Male 19 15 39
1 2 Male 21 15 81
2 3 Female 20 16 6
3 4 Female 23 16 77
4 5 Female 31 17 40

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
# Column Non-Null Count Dtype
---
0 CustomerID 200 non-null int64
1 Gender 200 non-null object
2 Age 200 non-null int64
3 Annual Income (k$) 200 non-null int64
4 Spending Score (1-100) 200 non-null int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
None
```

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

```
Dimensions de X : (200, 3)
Premières lignes standardisées :
[[-1.42456879 -1.73899919 -0.43480148]
 [-1.28103541 -1.73899919 1.19570407]
 [-1.3528021 -1.70082976 -1.71591298]
 [-1.13750203 -1.70082976 1.04041783]
 [-0.56336851 -1.66266033 -0.39597992]]
```

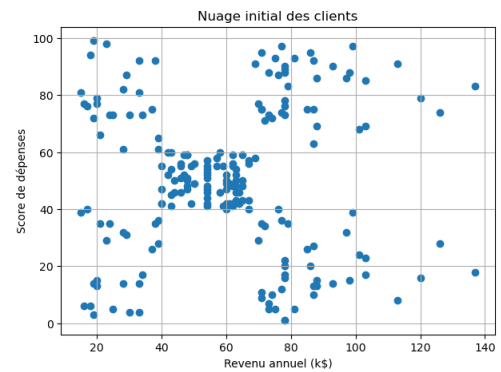
À ce stade, les données sont prêtes pour les algorithmes.

- Le tableau initial permet de vérifier que les colonnes attendues sont bien présentes.
- Le résumé statistique donne un premier aperçu des ordres de grandeur.
- Après standardisation, les variables sont placées sur des échelles comparables, ce qui est essentiel pour K-Means, la CAH et DBSCAN.

```
# Vérification
print("Dimensions de X :", X.shape)
print("Premières lignes standardisées :")
print(X_scaled[:5])
```

Une première visualisation dans le plan formé par le revenu annuel et le score de dépenses permet déjà d'observer une certaine structure dans les données.

```
# Nuage de points initial dans le plan revenu / score de dépenses
plt.figure(figsize=(7, 5))
plt.scatter(df["Annual Income (k$)"], df["Spending Score (1-100)"])
plt.xlabel("Revenu annuel (k$)")
plt.ylabel("Score de dépenses")
plt.title("Nuage initial des clients")
plt.grid(True)
plt.show()
```



Ce nuage de points suggère déjà que les clients ne sont pas répartis de manière totalement uniforme. On peut pressentir plusieurs zones plus denses, ce qui motive l'usage d'algorithmes de clustering.

Il faut toutefois rester prudent : à ce stade, il s'agit seulement d'une **impression visuelle** dans un plan à deux dimensions. Les algorithmes utiliseront ici trois variables : l'âge, le revenu annuel et le score de dépenses.

8.2. Mise en œuvre des méthodes non supervisées

8.2.1. K-Means

L'algorithme K-Means cherche à partitionner les observations en un nombre fixé de groupes homogènes. Chaque observation est affectée au cluster dont le centroïde est le plus proche. Comme le nombre de clusters n'est pas connu à l'avance, on peut s'appuyer sur la méthode du coude et sur le score de silhouette pour guider ce choix.

```
# =====
# 8.2.1. K-Means
# =====

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Liste des valeurs de K que l'on veut tester
K_values = range(2, 11)

# Listes pour stocker les résultats
inertias = [] # inertie = somme des distances au
centroïde
sil_scores = [] # score silhouette moyen

# Boucle sur plusieurs valeurs de K
for k in K_values:
    # Création du modèle K-Means
    kmeans = KMeans(
        n_clusters=k, # nombre de clusters
        init="k-means++", # meilleure initialisation des
        centroïdes
        random_state=42, # pour reproductibilité
        n_init=10 # nombre d'initialisations
    )
    # Apprentissage + attribution des labels
```

	K	Inertie	Silhouette
0	2	389.386189	0.335472
1	3	295.212246	0.357793
2	4	205.225147	0.403958
3	5	168.247580	0.416643
4	6	133.868421	0.428417
5	7	120.432833	0.411155
6	8	103.815256	0.408443
7	9	92.349458	0.420074
8	10	82.885266	0.422616

Meilleure valeur de K selon la silhouette : 6

Le tableau sert à comparer plusieurs valeurs de K.

- L'**inertie** diminue toujours lorsque K augmente, car les clusters deviennent plus spécialisés.
- Le **score de silhouette** est plus informatif pour le choix pratique de K, car il

```

labels = kmeans.fit_predict(X_scaled)
# Stockage de l'inertie
inertias.append(kmeans.inertia_)
# Calcul du score silhouette
sil_scores.append(silhouette_score(X_scaled, labels))

# Tableau récapitulatif
scores_df = pd.DataFrame({
    "K": list(K_values),
    "Inertie": inertias,
    "Silhouette": sil_scores
})

display(scores_df)

best_k = list(K_values)[np.argmax(sil_scores)]
print("Meilleure valeur de K selon la silhouette :",
      best_k)

```

tient compte à la fois de la compacité des groupes et de leur séparation.

En pratique, on cherche une valeur de K qui soit à la fois :

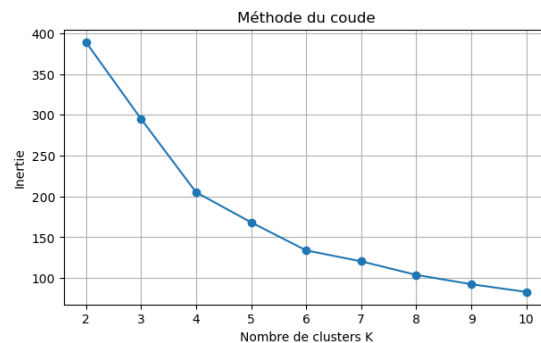
- raisonnable visuellement,
- cohérente avec la méthode du coude,
- et favorable selon le score de silhouette.

Méthode du coude : repose sur l'analyse de l'inertie en fonction du nombre de clusters.

```

plt.figure(figsize=(7, 4))
plt.plot(K_values, inertias, marker='o')
plt.xlabel("Nombre de clusters K")
plt.ylabel("Inertie")
plt.title("Méthode du coude")
plt.grid(True)
plt.show()

```



Sur cette courbe, on cherche une zone où la baisse de l'inertie commence à ralentir nettement. Ce changement de pente correspond au **coude**.

L'idée est la suivante : au-delà d'un certain nombre de clusters, ajouter de nouveaux groupes améliore encore l'inertie, mais le gain devient plus limité.

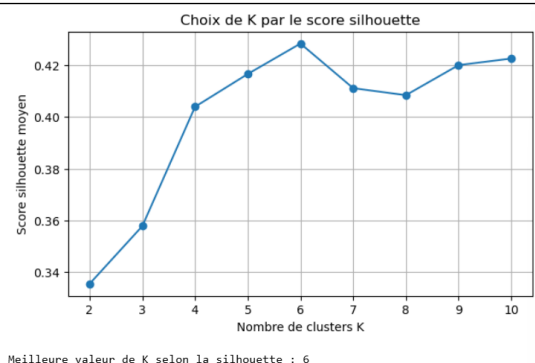
Score de silhouette : fournit un indicateur complémentaire de la qualité de la partition.

```

plt.figure(figsize=(7, 4))
plt.plot(K_values, sil_scores, marker='o')
plt.xlabel("Nombre de clusters K")
plt.ylabel("Score silhouette moyen")
plt.title("Choix de K par le score silhouette")
plt.grid(True)
plt.show()

# Valeur de K qui donne le meilleur score silhouette
best_k = K_values[np.argmax(sil_scores)]
print("Meilleure valeur de K selon la silhouette :", best_k)

```



Le score de silhouette varie entre -1 et 1.

- Une valeur proche de 1 indique un bon regroupement.
- Une valeur proche de 0 suggère des observations proches des frontières.
- Une valeur négative peut signaler une mauvaise affectation.

On peut retenir ici **K = 5**, car cette valeur est fréquemment utilisée pour ce dataset et conduit généralement à une segmentation interprétable.

Modèle final

Ici, pour rester cohérent avec l'usage fréquent de ce dataset, on peut choisir $K = 5$. Cela donne une segmentation interprétable de la clientèle.

```
# Création du modèle final
kmeans = KMeans(n_clusters=5, init="k-means++", random_state=42, n_init=10)

# fit_predict réalise deux opérations :
# 1) ajuste le modèle
# 2) renvoie directement le cluster de chaque observation
labels_kmeans = kmeans.fit_predict(X_scaled)

# On ajoute le résultat au DataFrame
df["Cluster_KMeans"] = labels_kmeans

# Aperçu
print(df[["Age", "Annual Income (k$)", "Spending Score (1-100)",
"Cluster_KMeans"]].head())

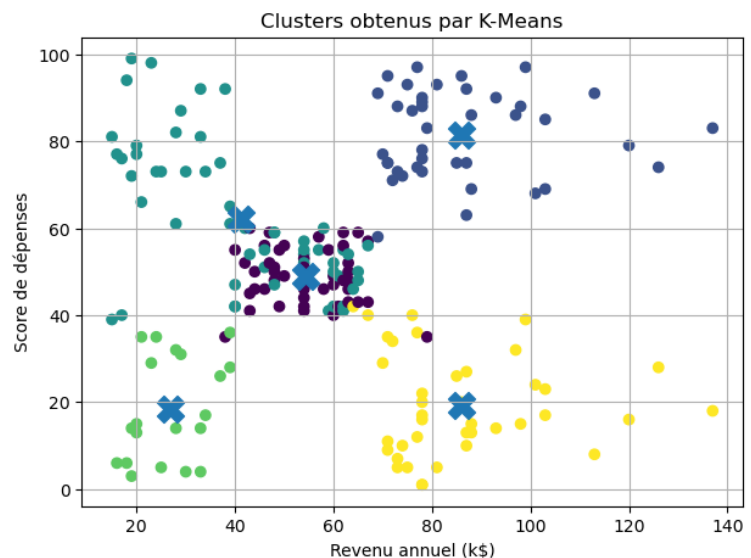
# Évaluation globale
print("Score silhouette final :", silhouette_score(X_scaled, labels_kmeans))
```

	Age	Annual Income (k\$)	Spending Score (1-100)	Cluster_KMeans
0	19	15	39	2
1	21	15	81	2
2	20	16	6	3
3	23	16	77	2
4	31	17	40	2

Score silhouette final : 0.41664341513732767

Visualisation des clusters : La représentation dans le plan revenu-dépenses permet de visualiser les groupes obtenus.

```
plt.figure(figsize=(7, 5))
plt.scatter(
    df["Annual Income (k$)"],
    df["Spending Score (1-100)"],
    c=labels_kmeans
)
plt.xlabel("Revenu annuel (k$)")
plt.ylabel("Score de dépenses")
plt.title("Clusters obtenus par K-Means")
plt.grid(True)
plt.show()
```



La figure met en évidence plusieurs groupes de clients. Les marqueurs en forme de **X** correspondent aux centroïdes, c'est-à-dire aux centres des clusters.

Dans ce plan, l'interprétation est souvent intuitive :

- certains groupes rassemblent des clients à **faible revenu et faible dépense** ;
- d'autres représentent des clients à **revenu élevé et forte dépense** ;
- d'autres encore traduisent des profils plus contrastés.

Cette visualisation est particulièrement utile dans une logique de **segmentation marketing**.

8.2.2. Clustering hiérarchique agglomératif

Le clustering hiérarchique construit progressivement une hiérarchie de groupes. Dans sa version agglomérative, chaque observation commence comme un cluster isolé, puis les groupes les plus proches sont fusionnés successivement. Le résultat est représenté par un dendrogramme.

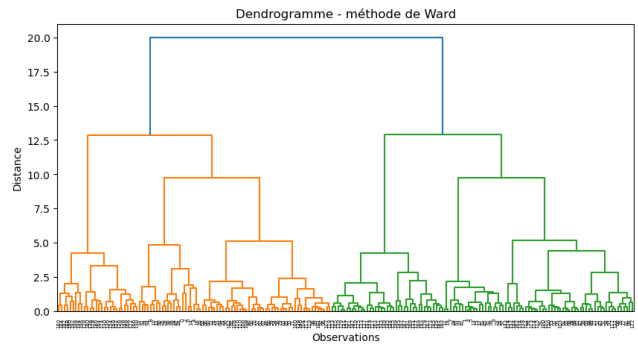
```
# =====
# 8.2.2. Clustering hiérarchique
# =====

from scipy.cluster.hierarchy import linkage, dendrogram
from sklearn.cluster import AgglomerativeClustering

# -----
# 1) Construction de la hiérarchie
# -----
# method="ward" : fusionne les groupes en minimisant
# l'augmentation de l'inertie intra-cluster
Z = linkage(X_scaled, method="ward")

# -----
# 2) Affichage du dendrogramme
# -----

plt.figure(figsize=(10, 5))
dendrogram(Z)
plt.title("Dendrogramme - méthode de Ward")
plt.xlabel("Observations")
plt.ylabel("Distance")
plt.show()
```



Le dendrogramme représente l'historique des fusions successives.

- Les fusions à **faible hauteur** correspondent à des groupes très proches.
- Les fusions à **grande hauteur** traduisent des regroupements plus coûteux.

En pratique, on choisit un niveau de coupe horizontal pour obtenir un nombre donné de clusters.

Ici, pour comparer avec K-Means, on retient également **5 clusters**.

Découpage en 5 clusters : Après examen du dendrogramme, on peut choisir un découpage en un nombre donné de clusters.

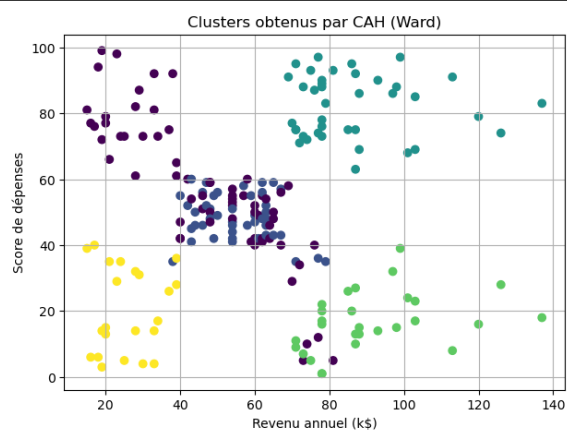
```
# On impose ici 5 clusters
hac = AgglomerativeClustering(n_clusters=5, linkage="ward")

# Attribution des clusters
labels_hac = hac.fit_predict(X_scaled)

# Ajout au DataFrame
df["Cluster_HAC"] = labels_hac
...
```

Visualisation : La partition obtenue peut ensuite être visualisée dans le même plan que précédemment.

```
plt.figure(figsize=(7, 5))
plt.scatter(
    df["Annual Income (k$)"],
    df["Spending Score (1-100)"],
    c=labels_hac
)
plt.xlabel("Revenu annuel (k$)")
plt.ylabel("Score de dépenses")
plt.title("Clusters obtenus par CAH (Ward)")
plt.grid(True)
plt.show()
```



La CAH produit ici une partition souvent proche de celle obtenue avec K-Means, mais son intérêt principal est différent:

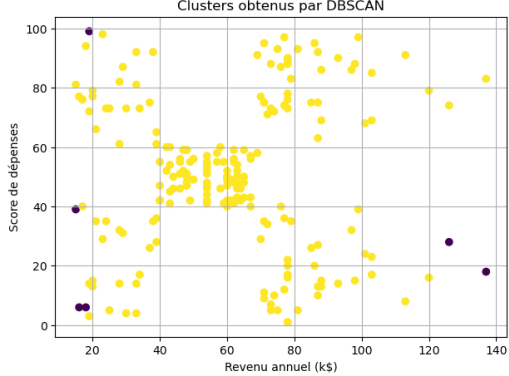
- elle donne accès à une **structure hiérarchique** ;
- elle permet de visualiser comment les groupes se forment progressivement ;
- elle ne force pas, dans un premier temps, le choix du nombre de clusters.

8.2.3. DBSCAN

DBSCAN est une méthode de clustering fondée sur la densité. Contrairement à K-Means, elle ne nécessite pas de fixer à l'avance le nombre de clusters, mais dépend de deux paramètres : `eps` et `min_samples`. Elle identifie les régions denses de l'espace et considère les points isolés comme du bruit.

<pre># ===== # 8.2.3. DBSCAN # ===== from sklearn.cluster import DBSCAN # eps : rayon du voisinage # min_samples : nombre minimum de voisins pour former un noyau dense dbscan = DBSCAN(eps=0.8, min_samples=5) # Attribution des labels labels_dbscan = dbscan.fit_predict(X_scaled) # Ajout au DataFrame df["Cluster_DBSCAN"] = labels_dbscan # Valeurs de labels trouvées print("Étiquettes trouvées :", np.unique(labels_dbscan)) # Les points de bruit ont le label -1 print("Nombre de points considérés comme bruit :", np.sum(labels_dbscan == -1))</pre>	Étiquettes trouvées : [-1 0] Nombre de points considérés comme bruit : 6 Avec DBSCAN, le label -1 correspond aux observations considérées comme du bruit. Ce sont des points qui n'appartiennent pas clairement à une région dense. Cette méthode est intéressante car elle : • peut détecter des clusters de forme non sphérique ; • ne nécessite pas de fixer K ; • identifie naturellement certains points atypiques
---	--

Visualisation : Le résultat peut être représenté graphiquement de la même manière.

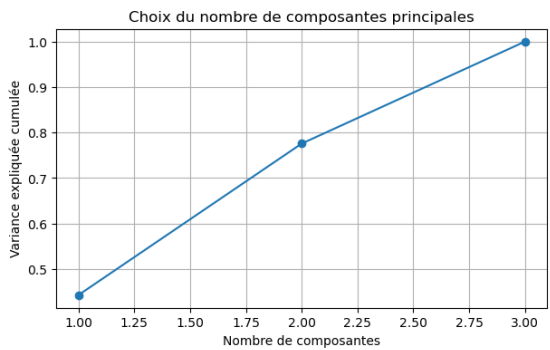
<pre>plt.figure(figsize=(7, 5)) plt.scatter(df["Annual Income (k\$)"], df["Spending Score (1-100)"], c=labels_dbscan) plt.xlabel("Revenu annuel (k\$)") plt.ylabel("Score de dépenses") plt.title("Clusters obtenus par DBSCAN") plt.grid(True) plt.show()</pre>	 Contrairement à K-Means, DBSCAN peut laisser certains points à l'écart sous forme de bruit. La figure permet donc de distinguer : • les régions denses identifiées comme clusters ; • les points plus isolés, qui peuvent être interprétés comme des cas atypiques. Le résultat dépend toutefois sensiblement du choix de <code>eps</code> et <code>min_samples</code>.
--	---

8.2.4. Réduction de dimensionnalité par ACP

L'Analyse en Composantes Principales permet de représenter les données dans un espace de plus faible dimension tout en conservant l'essentiel de l'information. Elle est particulièrement utile pour visualiser des données multidimensionnelles.

<pre># ===== # 8.2.4. ACP / PCA # ===== from sklearn.decomposition import PCA # ----- # 1) ACP complète # ----- # On calcule toutes les composantes principales pca = PCA() X_pca_full = pca.fit_transform(X_scaled) # Variance expliquée par chaque composante explained_var = pca.explained_variance_ratio_ # Variance cumulée cum_explained_var = np.cumsum(explained_var) print("Variance expliquée par composante :") print(explained_var) print("Variance expliquée cumulée :") print(cum_explained_var)</pre>	Variance expliquée par composante : [0.44266167 0.33308378 0.22425454] Variance expliquée cumulée : [0.44266167 0.77574546 1.] La variance expliquée mesure la part d'information capturée par chaque composante principale. ● La première composante porte la direction de variance maximale. ● La variance cumulée indique combien d'information est conservée lorsqu'on garde les premières composantes. Dans ce cas, comme on ne dispose que de trois variables initiales, l'analyse reste simple à interpréter.
--	---

Courbe de variance expliquée cumulée : permet d'apprécier le nombre de composantes nécessaires pour conserver une proportion suffisante de l'information.

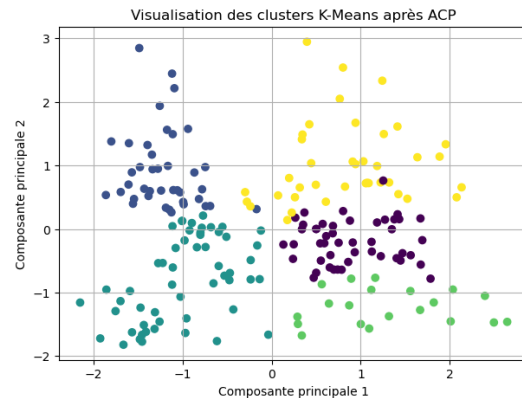
<pre>plt.figure(figsize=(7, 4)) plt.plot(range(1, len(cum_explained_var) + 1), cum_explained_var, marker='o') plt.xlabel("Nombre de composantes") plt.ylabel("Variance expliquée cumulée") plt.title("Choix du nombre de composantes principales") plt.grid(True) plt.show()</pre>	 Cette figure aide à choisir le nombre de composantes à conserver. Si deux composantes expliquent déjà une grande partie de la variance totale, alors une représentation en 2D devient pertinente pour la visualisation.
--	---

Projection en 2D : On peut projeter les données sur deux composantes principales.

<pre># On garde seulement 2 composantes principales pca2 = PCA(n_components=2) X_pca2 = pca2.fit_transform(X_scaled) print("Variance expliquée totale avec 2 composantes :", pca2.explained_variance_ratio_.sum())</pre>	Variance expliquée totale avec 2 composantes : 0.7757454566976748
---	--

Visualisation des clusters K-Means après ACP : Cette projection permet de visualiser les clusters obtenus par K-Means dans un espace réduit.

```
plt.figure(figsize=(7, 5))
plt.scatter(X_pca2[:, 0], X_pca2[:, 1],
            c=labels_kmeans)
plt.xlabel("Composante principale 1")
plt.ylabel("Composante principale 2")
plt.title("Visualisation des clusters K-Means après ACP")
plt.grid(True)
plt.show()
```



La projection ACP ne crée pas les clusters ; elle permet surtout de les **visualiser** dans un espace réduit.

Si les groupes restent bien séparés après projection sur deux composantes, cela confirme que la structure des données est relativement lisible.

Cette étape est donc très utile pour l'interprétation graphique des résultats.

8.2.5. Détection d'anomalies par Isolation Forest

La détection d'anomalies constitue une autre application importante de l'apprentissage non supervisé. Dans le cadre de la segmentation clients, elle permet d'identifier les comportements atypiques.

```
# =====
# 8.2.5. Détection d'anomalies
# =====

from sklearn.ensemble import IsolationForest

# contamination=0.05 signifie :
# on suppose qu'environ 5 % des observations sont
# atypiques
iso = IsolationForest(contamination=0.05,
                      random_state=42)

# fit_predict :
# 1 -> observation normale
# -1 -> anomalie
anomaly_labels = iso.fit_predict(X_scaled)

# Ajout au DataFrame
df["Anomaly"] = anomaly_labels

# Comptage
print(df["Anomaly"].value_counts())
```

```
1      190
-1      10
Name: Anomaly, dtype: int64
```

Les observations marquées -1 correspondent aux clients jugés atypiques par le modèle.

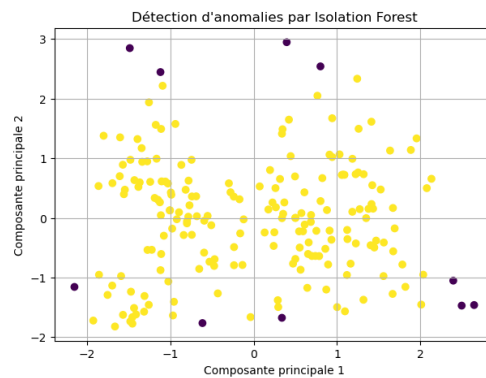
Il peut s'agir, par exemple, de profils présentant une combinaison inhabituelle entre l'âge, le revenu et le score de dépenses.

Ce type d'analyse peut être utile pour :

- détecter des comportements rares ;
- cibler des profils particuliers ;
- compléter une segmentation classique par une étude des cas extrêmes.

Visualisation dans l'espace ACP

```
plt.figure(figsize=(7, 5))
plt.scatter(X_pca2[:, 0], X_pca2[:, 1],
            c=anomaly_labels)
plt.xlabel("Composante principale 1")
plt.ylabel("Composante principale 2")
plt.title("Détection d'anomalies par Isolation Forest")
plt.grid(True)
plt.show()
```



Dans cette figure, les points atypiques apparaissent séparés des régions les plus denses. La projection ACP aide ici à voir rapidement si les observations détectées par Isolation Forest se situent en marge de la structure principale.

Il faut néanmoins garder à l'esprit que cette visualisation est une projection en 2D d'un traitement effectué sur les données standardisées complètes.

Extraction des clients atypiques

```
clients_atypiques = df[df["Anomaly"] == -1]
print(clients_atypiques.head())
```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)	\
0	1	Male	19	15	39	
2	3	Female	20	16	6	
8	9	Male	64	19	3	
10	11	Male	67	19	14	
30	31	Male	60	30	4	

	Cluster_KMeans	Cluster_HAC	Cluster_DBSCAN	Anomaly
0	2	4	-1	-1
2	3	4	-1	-1
8	3	4	0	-1
10	3	4	0	-1
30	3	4	0	-1

Ces observations peuvent correspondre à des clients présentant un comportement de consommation inhabituel par rapport à l'ensemble de la population étudiée.

8.3. Bilan

Cette implémentation met en évidence le déroulement classique d'une analyse non supervisée en Python. La première étape consiste à préparer les données, notamment en sélectionnant les variables pertinentes et en les standardisant. Ensuite, plusieurs méthodes complémentaires peuvent être mobilisées selon l'objectif recherché :

- **K-Means** permet une segmentation simple, rapide et facilement interprétable ;
- **le clustering hiérarchique** ajoute une lecture structurée des regroupements grâce au dendrogramme ;
- **DBSCAN** détecte les groupes denses et les observations isolées sans imposer le nombre de clusters ;
- **l'ACP** facilite la visualisation et la réduction de dimensionnalité ;
- **Isolation Forest** repère les observations atypiques.

✉ **Fil conducteur — Segmentation client** : appliquées au jeu de données Mall Customers, ces méthodes permettent de faire émerger différents profils de clients à partir de l'âge, du revenu annuel et du score de dépenses. L'analyse obtenue peut servir à la segmentation marketing, au ciblage d'offres ou encore à l'identification de comportements inhabituels.